CEPARCO Project Proposal

Group 6 Members:

1. Bactong, Gabrielle Joei V.
2. Lobo, Shirley Marie A.
3. Panganiban, Diane
4. Gunita, Catherine Rosswyn D.

01. Abstract

The SVM algorithm is widely used in machine learning training. It is utilised in classification tasks and regression analysis. The aim of this project will be to adopt the study by Suarez et al. (2025), which focuses on optimising SVM using a linear regression approach. The same dataset used by Suarez et al. (2025) will be utilised in this project. The group will use YMM SIMD registers to implement the equations used in the linear regression approach. This addresses time complexity from executing barycenter (geometric) equations and criteria. In this study, all processes aside from data reduction such as linear optimization problem 7, point-convex hull test, misclassification, soft and hard margin will remain sequential. The expected output of the project will be the reduced and filtered training dataset, performance and execution time speedup matrices of the parallel approach, as well as the correctness check of the training dataset filtered through SIMD. A C implementation will also be implemented for the purpose of comparison with the resulting matrices of the parallel approach.

02. Description of Proposed Project

Support Vector Machines (SVMs) are widely used in machine learning training. They are used for classification tasks and regression analysis that can help with image classification, fraud detection, text classification, recommender systems, etc (Mun, 2023). However, the time it takes to train SVMs scales significantly with the number of data points N. Every data point is a decision variable, causing SVMs have a time complexity of $O(N^3)$ (Abdiansah & Wardoyo, 2015; Suárez et al., 2025). A study by Suárez et al. (2025) approached this problem by removing points through linear programming that have no chance to be support vectors, reducing N and speeding up training. Thus, our group aims to build off their research and to parallelize the remaining stages in their SVM pipeline, combining linear programming and SIMD programming to optimize training.

a. Inputs to the project

The main input to be used for the project is the same training dataset used by Suárez et al (2025). The dataset to be used is labeled dataTra.xlsx and is an $N \times D$ matrix with columns $x1$, $x2$, $x3$, $x4$, x5, and labels y. The dataset is accessible through this GitHub link: Dataset.

b. Proposed Process

i. Processes to be parallelized

The processes to be parallelized using AVX2 are the following: (1) Barycenters (Geometric) Calculation, and (2) filtering of the barycenters.

ii. Existing Implementations

Over the years, attempts to optimize SVM algorithms have been proposed and studied by scholars. The identified implementations are honourable mentions from different approaches. Among the published studies, the linear regression method for optimising SVM, such as the method proposed by Suarez et al. (2025), has become the focus of our project. It essentially integrates linear programming techniques with SVM to classify optimal separators between sets of points. Another implementation is an optimisation-focused approach by Woodsend, K., & Gondzio, J. (2011), in which they introduce an interior point method for linear SVM training which takes advantage of the Hessian separability. The geometric approach using the convex hull is also a popular method of optimising the SVM algorithm. A study conducted by Mavroforakis ME and Theodoridis S (2006) introduced the concept of reduced convex hull and adapted the Schlesinger-Kozinec algorithm to be applied to both separable and non-separable SVMs. Lastly, clustering-based data reduction approach which removes data points that are unlikely to become a support vector (Suarez et al., 2025). An example study part of this approach is by Shen et al. (2016). Which introduced the usage of parallel hyperplane scheme which combines k-means clustering with recursive hyperplane formation, effectively eliminating redundant points.

iii. Plans on how to parallelize the proposed process

As mentioned, the proposed project will build upon the implementation presented by Suarez et al. (2025). The project will optimize selected computationally intensive stages of the SVM data-reduction pipeline while retaining the remaining stages of the original workflow. In this project, the group will be optimizing the time complexity from performing barycenter (geometric) equation and the barycenter criteria by implementing the following equations using AVX2 SIMD instructions:

Dimension Reduction Heuristics:

I. Barycenter (Geometric) Calculation

$$z^{B_1} = \frac{1}{m}\sum_{i=1}^{m} x^i, \tag{8}$$

$$z^{B_2} = \frac{1}{n}\sum_{j=1}^{n} y^j. \tag{9}$$

Figure 3: Barycenter (Geometric) Calculation Equation (image from: Suarez et al. (2025))

The barycenter calculation calculates the center point of different classes and identifies if the point in space is within the validity range specified by the Barycenter criteria. x86-64 assembly with AVX2 SIMD instructions and 256-bit YMM registers can be used to parallelize geometric barycenter filtering. These

instructions will allow multiple numerical values to be processed within a single vector operation by loading 8 single-precision float values per instruction using SIMD arithmetic operations.

II. **Barycenter Criteria**

$$\|x - z^{B_j}\|_2 > \|z^{B_i} - z^{B_j}\|_2, \quad i \neq j. \tag{10}$$

Figure 4: Barycenter Criteria Equation (image from: Suarez et al. (2025))

The Barycenter Criteria can be parallelized by performing the euclidean distance operation to find the distance between the two identified barycenters and the distance between point X against a barycenter point. This utilizes the YMM register by loading 8 single precision floats per instruction. Where it performs SIMD arithmetic operations for calculating the Euclidean distance. For comparison, the compare packed instructions will be utilized to check if the point is within the valid area. Otherwise, the point will be disregarded.

Correctness Checking:

The group will be implementing their own sequential version of the SVM pipeline using C due to its pointers and arrays. Stages 1 to 3 of the pipeline (sequential barycenter calculations and filtering and gram matrix evaluation) will be implemented using arithmetic functions from math.h and nested for loops for dot products. Meanwhile, linear programming checks (LP 7 and LP 11-13) will be implemented using open-source libraries such as HiGHS or GLPK, intended for large-scale linear programming (LP) models. C will be the main driver that will be used to call on the SIMD implementation to compare speedup and correctness with the sequential implementation.

iv. Other processes in the project that will not be parallelized

Aside from the processes mentioned above, the point-in-convex hull test, linear optimization problem 7 feasibility check, misclassification, as well as the soft and hard margin accomplished by Suárez et al. (2025) will not be parallelized and will remain sequential.

c. Outputs of the project

The final output of the project will be the reduced and filtered training dataset. Additionally, the project will output the performance and execution time speedup metrics of the parallel approach will be compared to the sequential approach programmed in C ($T = \frac{T_{sequential}}{T_{parallel}}$). Other outputs include the correctness or sanity check of the training dataset filtered through SIMD.

03. Proposed target implementation platform of the proposed project

Development will be conducted using Visual Studio Code as the primary code editor, GCC or MSVC for compiling C code, NASM for assembling x86-64 assembly routines. The implementation will run on a system with an AVX2-compatible CPU. The final toolchain, operating system, and hardware specifications will be documented once confirmed.

A sequential C implementation will serve as the baseline for performance comparisons, and the reference for validating the correctness of the parallel implementation. All implementations will use equivalent input datasets and workloads. Performance will be evaluated using execution time and speedup, calculated by dividing the sequential C execution time by the parallel implementation's execution time. Correctness checks will account for acceptable floating point differences and vector boundary handling.

04. Mid-project milestones

The timeline below serves as our mid-project milestones for the proposed project:

Milestone #1: Week 7-8: Baseline implementation of the sequential C program.

- Sequential implementation for the following processes: barycenters (Geometric) calculation, and filtering of the barycenters.

Milestone #2: Week 8-9: SIMD implementation of barycenters calculation and filtering

Milestone #3: Week 10: Mid-project update demo

The continuation serves as the remaining milestones for the project:

Milestone #4: Week 10-11: Unit testing, bug hunting and finalizing

Milestone #5: Week 11-12: Testing of correctness, performance, and speedup metrics

Milestone #6: Week 13: Complete project submission and demo

05. Mid-project milestone update

The following features were implemented as completion of the mid-project milestone:

1. **Data extraction of feature values and classes from a synthetic dataset.**

	A	B	C	D	E	F
1	**x1**	**x2**	**x3**	**x4**	**x5**	**y**
2	19.79	19.29	0	441	0.002746	-1
3	20	18.7	0	438	0.002696	-1
4	19.745	19.2675	0	451.75	0.002735	-1
5	19.79	19.1	0	435	0.002718	-1
6	19.79	19.29	0	441	0.002746	-1
7	20	19.34	116	453	0.002789	-1
8	19.92667	18.79	0	439.6667	0.002697	-1
9	20	18.79	0	441	0.002709	-1

Fig 1. Structure of sample.xlsx file

- For testing purposes, a smaller dataset, sample.xlsx (containing 815 rows), was used instead of dataTra.xlsx (containing 8144 rows). The .xlsx file was converted to a .csv file for easier data extraction.
- Dynamically allocated 2D arrays, `X_classNeg1` and `X_class1`, are initialized to store datapoints of classes -1 and 1, respectively.
- `fgets()` was used to read the .csv line by line until the first '\n'. Each line was tokenized using `strtok()` based on the delimiter ','.
- For each token, all feature values (x1, x2, x3, x4, x5) are extracted and converted to floats via `(float)atof()`. These feature values are initially stored as unclassified data points before being stored in their respective classes, `X_classNeg1` or `X_class1`.
- **The result:** Dynamically allocated 2D arrays `X_classNeg1` and `X_class1` containing all datapoints read from sample.csv belonging to class -1 and 1, respectively. These populated 2D arrays are needed for barycenter calculations.

2. **Sequential implementation of barycenters calculation.**

$$z^{B_1} = \frac{1}{m} \sum_{i=1}^{m} x^i,$$

Fig 2. Barycenter calculation equation

- A barycenter is the geometric center of all data points belonging to a specific class (i.e., -1, 1). Using a nested for loop, the algorithm processes the 2D array (`X_classNeg1` or `X_class1`) column by column across all five features (x1, x2, x3, x4, x5)
- For a given class, the algorithm iterates down all m rows and sums the values of the current feature column.
- Once a column is fully traversed, the total feature sum is divided by the number of points m to obtain the geometric center of that feature. This is repeated for all five features.
- **The result:** A 5D vector containing the geometric centers of features x1, x2, x3, x4, and x5 of that class.

3. **SIMD implementation of barycenters calculation.**

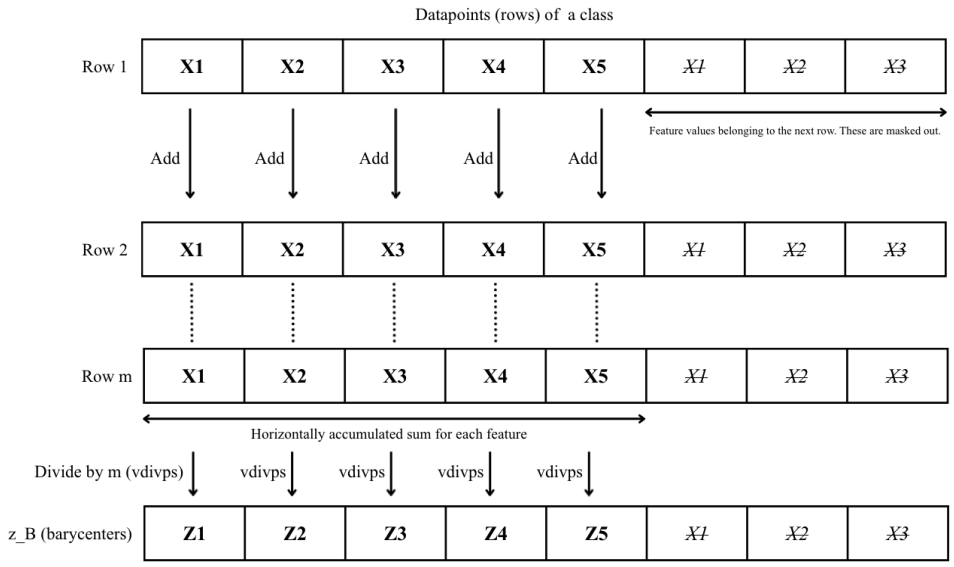


Fig 3. Diagram of SIMD implementation of calculating for barycenters

- Instead of iterating through the 2D array column by column across all five features, SIMD YMM registers were used to horizontally iterate down all m rows for each feature (x1, x2, x3, x4, x5) simultaneously.
- 256-bit YMM registers can store 8 floats and risk reading values belonging to another datapoint. `vmaskmovps` was used to mask only the 5 values belonging to the correct datapoint.
- Once all datapoints have been fully traversed, the total sum for each feature is divided by the number of points m to obtain the geometric center of all the datapoints.
- **The result:** A SIMD calculated 5D vector containing the geometric centers of features x1, x2, x3, x4, and x5 of that class.

4. SIMD implementation of Barycenter Criteria

- Given that the dataset is a 5D dataset, we used the vmaskmovps to mask only the 5 features of the given datapoint to prevent unwanted values from other datapoints.
- The program iterates through the loop of operation for each datapoints (row), it includes operations such as subtraction (vsubps), multiplication (vmulps), addition (vaddps and vhaddps), square root (vsqrtss), and a conditional statement that checks if the given datapoint is an outlier. An outlier is determined when the calculated euclidean distance between a given point of class X is further than the euclidean distance of class X and class Y. If true, then the datapoint is not stored into the retained vector of valid datapoints.
- **The result:** A SIMD calculated set of valid 5D vectors containing valid datapoints that meets the criteria condition and the total number of valid points.

5. Sequential C implementation of Barycenter Criteria

- The function calculate_barycenter_criteria_c() implements the sequential filtering of data points using the Barycenter Criteria.
- Initially, the program calculates the Euclidean distance between the two class barycenters (own_barycenter and opposite_barycenter) using a loop that iterates through all features.

- The program then iterates through each datapoint in the input array X and calculates its Euclidean distance from the opposite class barycenter.
- Using sqrtf() from math.h, the squared differences between corresponding features are summed and converted into Euclidean distances.
- A conditional statement checks whether the calculated distance of a datapoint from the opposite barycenter is less than or equal to the distance between the two class barycenters.
- If the condition is satisfied, the datapoint is copied into the filtered array. Otherwise, the datapoint is disregarded.
- The program continues until all datapoints have been evaluated. A counter (retained_rows) keeps track of the total number of retained datapoints.
- **The result:** A filtered dataset containing valid datapoints that satisfy the Barycenter Criteria, along with the total number of retained rows.

06. References

Abdiansah, A., & Wardoyo, R. (2015). Time Complexity Analysis of Support Vector Machines (SVM) in LibSVM. International Journal of Computer Applications, 128(3), 28–34. https://doi.org/10.5120/ijca2015906480

Mavroforakis ME, Theodoridis S (2006) A geometric approach to support vector machine (svm) classification. IEEE Trans Neural Netw 17(3):671–682. https://doi.org/10.1109/TNN.2006.873281

Mun. (2023, February 19). SVMs in Practice: Applications and use cases for machine learning's most effective model. Medium. https://medium.com/@mun.articles/svms-in-practice-applications-and-use-cases-for-machine-learnings-most-effective-model-2ae25f4207ef

Woodsend, K., & Gondzio, J. (2011). Exploiting separability in large scale linear support vector machine training. Computational Optimization and Applications, 49(2), 241–269. https://doi.org/10.1007/s10589-009-9296-8

Shen, X.-J., Mu, L., Li, Z., Wu, H.-X., Gou, J.-P., & Chen, X. (2016). Large-scale support vector machine classification with redundant data reduction. *Neurocomputing*, *172*, 189–197. https://doi.org/10.1016/j.neucom.2014.10.102

Suárez, C. A., Castro, M., Leon, M., Martin-Barreiro, C., & Liut, M. (2025). Improving SVM performance through data reduction and misclassification analysis with linear programming. Complex & Intelligent Systems, 11(8), 356. https://doi.org/10.1007/s40747-025-01989-4